

SIH 2026 — Which Theme to Keep, and Why

A decision note on SentinelAI and MachineSense AI for Smart India Hackathon 2026. Submission deadline: 20 September 2026. 226 problem statements published; 500 idea slots per statement.

The decision

Project	Theme to keep	Problem statement
SentinelAI	Smart Automation	SIH26187 - Ministry of Home Affairs. AI video analytics over existing CCTV.
MachineSense AI	Robotics and Drones	SIH26054 - DRDO. Digital twin for UAV aero piston engines.

Start here: why the pitch has to change

SIH 2026 has no women-safety problem statement and no MSME predictive-maintenance problem statement. I checked all 226. This matters because both of your projects were written for a different competition. SentinelAI was written for the "Sovereign Technology for India" challenge, where the judging cares about self-reliance, budget and patents. MachineSense was written for the MSME Idea Hackathon, where the judging cares about affordability and business model.

SIH does not work that way. In SIH a named government or public-sector organisation writes the problem, states exactly what the software must do, and evaluates your entry against that list. There is no credit for market size, project cost or patent potential. There is heavy credit for building the specific capability they asked for and showing it working.

So the question is not "which theme is closest to my project". It is "which organisation is asking for a system I have already largely built". Judged that way, each of your projects has one clear answer.

How the two choices were made

I scored every one of the 226 statements against what your code actually does, not what your slides claim. For SentinelAI that meant looking for statements about turning existing cameras into an active detection system with real-time alerts. Four candidates came up. For MachineSense it meant looking for statements about equipment health, anomaly detection, remaining useful life or digital twins. Three candidates came up. The two recommended below are the ones where the least new work is needed for the most complete answer.

The two also sit in different themes on purpose. Many colleges cap how many entries one theme may carry from a single institution, and internal shortlisting often compares entries within a theme. Splitting SentinelAI and MachineSense across two themes means they never compete against each other.

1. SentinelAI - keep the theme Smart Automation

SIH26187, Ministry of Home Affairs. "AI-Based Intelligent Video Analytics Platform for Border Surveillance using existing CCTV Infrastructure." Software category.

What the Ministry is asking for

Border security forces already have CCTV cameras at Border Out Posts, check posts and border roads. Those cameras only record; a person has to sit and watch them. Buying smart cameras with built-in face recognition or number-plate recognition is expensive and hard to deploy in remote areas. So MHA wants a software layer that takes the video from the ordinary IP cameras they already own and makes it intelligent.

The capabilities they list are: human detection and tracking, vehicle detection and classification, face detection, automatic number-plate recognition, virtual-fence intrusion detection, suspicious-activity detection, night-time movement detection, real-time alert generation with event logging, and integration with their existing command and control system. The solution must be cost-effective, scalable and deployable in remote areas.

Why this is SentinelAI

Read your own Q1 and Q2 answers again. Your central argument is that CCTV cameras only record instead of acting, that emergency systems require manual triggering, and that existing public infrastructure is strategically placed but underused. MHA is making the identical argument about border cameras. Your answer was to add local AI inference that detects, validates and alerts without a human in the loop. That is precisely what they are asking someone to build.

The only part of SentinelAI that does not transfer is the physical layer - the smart pole, the microphone, the vibration sensor, the illumination control. Everything above that layer transfers directly.

What you keep, and how to describe it

Your component	How to present it for SIH26187
Computer vision stack (YOLO, OpenCV, TensorFlow)	Unchanged. This is the core deliverable. Detection classes change to person, vehicle, face and plate.
Multi-sensor validation engine	Rename to multi-cue validation: confirm an event across consecutive frames and across neighbouring cameras before raising an alert. Sell it as the false-alarm killer - control rooms abandon systems that cry wolf.
Safe Corridor (poles collaborating)	Rename to cross-camera handoff: when a person or vehicle leaves camera A's field of view, camera B picks up the same track so the trail never breaks. This directly answers their "object tracking" requirement.
Edge AI processing	Keep and emphasise. Border posts have poor connectivity, so on-site inference is a requirement here, not a bonus feature.
Alert pipeline and GPS tagging	Keep. Becomes real-time alert generation plus event logging, with a documented interface for their command and control system.
React + Node dashboard	Keep as the operator console: live camera grid, alert feed, event search, virtual-fence editor.

What you remove, and why

Remove the smart poles, streetlights, microphones and vibration sensors. The statement is Software category and explicitly says the platform must work over standard IP cameras with no dedicated hardware - proposing hardware argues against the problem they set.

Remove the women's-safety and family-notification material. It is not the user group here, and it makes the entry read as a repurposed submission, which evaluators notice.

Remove the Rs. 18-20 lakh budget, the patent opportunities and the market and audience sections. SIH scores the solution and its feasibility. Investment material takes space away from the technical content that is actually marked.

What to build before you submit

First, number-plate recognition that genuinely works on Indian plates in poor light and at an angle. This is the single hardest item on their list and the one where most teams under-deliver, so it is where you can separate yourself. Second, a virtual-fence drawing tool in the dashboard - it demos in ten seconds and proves the alerting path end to end. Third, one measured false-alarm number from a real test video, because a measured figure beats any claim in a deck.

If time is short, cut face recognition before you cut number plates. Face recognition at border distances is hard to demo convincingly; plate reading is not.

2. MachineSense AI - keep the theme Robotics and Drones

SIH26054, DRDO. "AI-Enabled Real-Time Digital Twin System for Health Monitoring, Fault Prediction and Mission Reliability Enhancement of Aero Piston Engines used in MALE UAVs." Software category.

What DRDO is asking for

Medium Altitude Long Endurance UAVs fly long surveillance and reconnaissance missions. If the piston engine fails mid-flight the mission is aborted and the aircraft may be lost. Today's engine monitoring is threshold-based: it tells you something is wrong only after it has already gone wrong. DRDO wants a digital twin - a live virtual copy of the engine, fed by sensor data and physics models, that spots degradation early, predicts failures before they happen, estimates remaining useful life, and lets an operator simulate how the engine would behave on a given mission profile.

Why this is MachineSense

Of all 226 statements this is the only one that asks for your exact combination of features in a single system. Everything in the table below already exists in your backend.

What DRDO asks for	What you already have in ml.py
Monitoring of engine health indicators	Weighted 0-100 health score computed from normalised sensor deviations
Detection of abnormal operating conditions	Median-absolute-deviation z-score anomaly detection against the unit's own recent history
Estimating degradation trends and remaining useful life	Least-squares slope of the health trend extrapolated to a failure threshold
Predicting probable failures before they occur	Logistic failure probability from health level and degradation speed
Simulating engine behaviour under different mission profiles	Digital twin what-if engine that projects health, life and consumption after an action
Real-time parameter visualisation	Live dashboard with gauges, telemetry charts and before-and-after twin comparison
Justifying the assessment to an operator	Explainable-AI layer attributing the health penalty to each sensor in plain language

Your simulator is an advantage here

You have no real engine data and you will not get any. That would sink most entries. It does not sink this one, because the statement asks for a virtual representation driven by sensor data, thermodynamic models and performance maps - a simulated engine state is an accepted way to demonstrate it. Keep simulator.py, but re-label its channels as engine parameters: cylinder head temperature, exhaust gas temperature, RPM, manifold pressure, fuel flow, oil pressure, vibration.

What you remove, and why

Remove the entire MSME framing. The customer here is a defence research organisation, not a small factory owner, and affordability is not a judging criterion.

Remove the energy optimisation module with its rupee tariff and CO2 factors. It is tuned to Indian grid electricity and has no meaning for an airborne engine. If you want to keep an efficiency angle, convert it to fuel consumption and specific fuel consumption instead.

Remove the market size, SaaS subscription model, hardware kit and the Rs. 19.8 lakh project cost. Same reason as SentinelAI: not scored, and it dilutes the technical story.

The one thing that will decide your score

DRDO asks for a physics-informed digital twin. They name thermodynamic behaviour models and engine performance maps as inputs alongside the AI. Your current twin is statistical: applying an action reduces sensor severities and the health score is recomputed. A judge who knows engines will read that as a dashboard with a what-if slider, not a digital twin, and that single judgement can cost you the round.

The fix does not have to be heavy. A simple thermodynamic relationship linking RPM, manifold pressure and fuel flow to expected cylinder head and exhaust gas temperatures, with your AI learning the residual between predicted and observed values, is enough to make it a genuine hybrid twin. The residual then becomes your degradation signal, which is also a stronger anomaly detector than a raw threshold. Add mission replay - re-run a recorded flight through the twin - and you have answered the statement fully.

3. The fallback, if the physics work is too heavy

SIH26170, ISRO. "AI-Driven Anomaly Detection in Component Burn-In and Screening." Software category, Smart Automation theme.

Electronic components for space are stress-tested at high temperature for long periods. Some parts pass the pass/fail limits but drift suspiciously over time, and those latent defects end up in satellites and fail in orbit. ISRO wants a model that catches them.

They ask for three things and you have all three already. Module A is dynamic outlier detection - a part reading 45 microamps in a lot averaging 10 is an anomaly even though the datasheet limit is 50 - which is exactly what your MAD z-score does. Module B is a regression that takes the 0-hour and 24-hour readings and forecasts the 168-hour value, flagging parts whose predicted drift is too steep, which is your least-squares extrapolation with a shorter horizon. The third is explainability, listed as an explicit evaluation metric - can the model justify its call to a QA inspector - and that is the layer you already built and that almost no competing team will have.

The trade-off is that this statement sits in Smart Automation, the same theme as SentinelAI's SIH26187. Take it only if your college does not cap entries per theme, or if you decide to move SentinelAI to the Qualcomm environmental-monitoring statement (SIH26178, Disaster Management) instead.

One tuning note if you take it: weight false negatives much more heavily than false positives in your loss function. ISRO says outright that letting a bad part escape is catastrophic, so a model that rejects a few good parts will score better than one that misses a defective one.

Summary of what to do next

For SentinelAI, rewrite the problem framing from public emergency response to border surveillance, delete every hardware and business section, and spend your remaining build time on number-plate recognition, the virtual-fence tool and a measured false-alarm figure.

For MachineSense, relabel the simulator channels as aero engine parameters, delete the MSME, energy-cost and business material, and add a lightweight physics layer plus mission replay so the twin reads as a twin.

Both entries are due 20 September 2026. The technical work above is the deliverable; the writing is a day. Do not spend the month on the deck.